

Unidad 1: Concepto de NoSQL

Clase 1 – Distribución, consistencia y disponibilidad en sistemas de datos

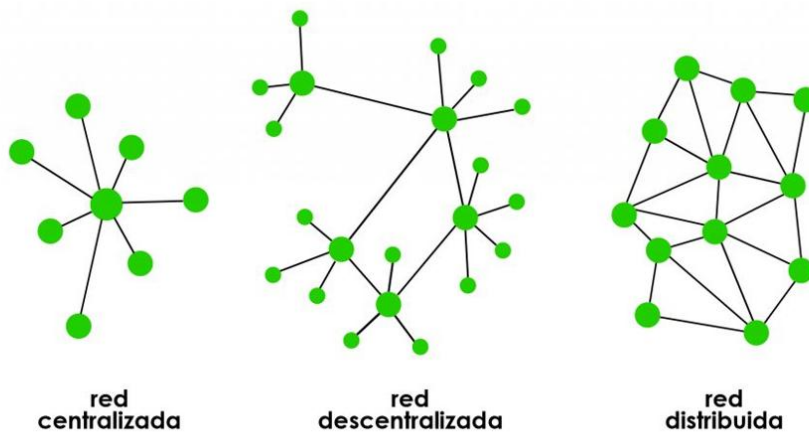
Objetivo

Analizar cómo la distribución y replicación de datos impactan en la consistencia y disponibilidad de los sistemas, introduciendo los modelos de consistencia y los principios que luego serán formalizados mediante el teorema CAP.

Tema

- Distribución de datos.
- Replicación y particionamiento (introducción).
- Modelos de consistencia.
- Disponibilidad vs consistencia.
- Introducción al teorema CAP (sin formalizar aún).

¿Qué significa distribuir datos?



Comparación entre una arquitectura centralizada (izquierda), descentralizada (centro) y distribuida (derecha). En un sistema distribuido, múltiples nodos (puntos verdes) almacenan y gestionan partes de los datos, colaborando sin depender de un único servidor central.

Distribuir datos significa almacenar y procesar la información en múltiples ubicaciones o nodos interconectados, en lugar de concentrarla en un solo servidor central. En una base de datos **centralizada**, todos los datos residen en un único servidor o ubicación física, lo que simplifica la administración pero crea un punto único de fallo (si ese servidor cae, todo el sistema se ve afectado). En cambio, en una base de datos **distribuida**, los datos se **reparten entre varios servidores** ubicados en diferentes lugares, conectados por una red. Cada nodo del sistema distribuido mantiene una

porción (o copia) del conjunto total de datos y opera de forma **autónoma**, pero coordinada con los demás. Para el usuario o las aplicaciones, el sistema distribuido funciona *como si* fuera uno solo – es decir, el acceso a los datos es transparente, sin importar en qué servidor físico se encuentren. En resumen, *distribuir los datos* implica tener varios equipos (**nodos**) colaborando para almacenar y gestionar la información, presentando lógicamente una base de datos unificada.

¿Por qué se distribuyen los datos?

Existen **varias razones clave** por las que se opta por distribuir los datos en lugar de centralizarlos en un único sitio. Principalmente, este enfoque busca mejorar el **rendimiento**, lograr una mayor **escalabilidad** y aumentar la **resiliencia** o tolerancia a fallos del sistema. A continuación, se explican estas ventajas en detalle:

- **Mejor rendimiento:** Un sistema distribuido puede procesar grandes volúmenes de información de forma más rápida y eficiente. Al dividir las tareas o las consultas entre múltiples nodos, se logra un **procesamiento en paralelo** que reduce el tiempo total de cómputo y la **latencia** percibida por los usuarios. Por ejemplo, en lugar de que un solo servidor atienda todas las solicitudes, varios servidores pueden trabajar simultáneamente en diferentes partes de la carga de trabajo. Además, la distribución permite colocar los datos *más cerca de los usuarios o de los puntos de consumo*, minimizando retrasos de red. Esto significa que los usuarios pueden acceder a la información de manera más ágil según su ubicación geográfica, mejorando los tiempos de respuesta del sistema. En conjunto, estas características elevan el **rendimiento** global al evitar cuellos de botella y aprovechar la potencia combinada de múltiples máquinas.
- **Escalabilidad horizontal:** La distribución de datos facilita expandir la capacidad del sistema de forma **incremental** y económica a medida que crecen las necesidades. En lugar de depender de un único servidor cada vez más potente (escalado vertical), se pueden **añadir más nodos** o servidores en la red para repartir la carga de trabajo. Esta **escalabilidad horizontal** permite manejar volúmenes de datos y números de usuarios crecientes sin una degradación significativa del rendimiento. Por ejemplo, si una empresa necesita procesar más datos, puede simplemente integrar otro servidor a la base de datos distribuida, aumentando así la capacidad de almacenamiento y cómputo del sistema. Esto proporciona una solución flexible: el sistema puede crecer de manera prácticamente transparente, manteniendo la eficiencia operativa al distribuir automáticamente las nuevas cargas entre los nodos disponibles. En definitiva, los sistemas distribuidos *"pueden crecer a medida que la empresa lo hace"*, evitando los límites de un solo equipo y adaptándose dinámicamente a la demanda.
- **Resiliencia y alta disponibilidad:** Una motivación crucial para distribuir datos es mejorar la **resiliencia**, es decir, la capacidad de seguir funcionando pese a fallos parciales. Al tener la información replicada o particionada en múltiples servidores, el sistema no depende de un único punto cuya falla lo colapse. Si **un nodo falla**, los demás pueden continuar operando y ofreciendo acceso a (al menos parte de) los datos, lo que **evita una caída total** del servicio. De este

modo, se incrementa la **tolerancia a fallos**: el sistema en conjunto sigue disponible aunque ocurran problemas en alguno de sus componentes. Por ejemplo, en una base de datos distribuida, la caída de un servidor no inutiliza la base de datos completa, ya que otras réplicas pueden atender a los usuarios. Esto contrasta con un esquema centralizado donde una avería deja inaccesible *toda* la información. Como resultado, los sistemas distribuidos logran una **alta disponibilidad** y confiabilidad mayores. De hecho, “la falla de un único servidor no afecta a todo el sistema” en un entorno distribuido, permitiendo continuidad de las operaciones. Esta propiedad es vital para aplicaciones críticas (bancos, comercio electrónico, etc.), que no pueden permitirse puntos únicos de falla. Además, la distribución geográfica de datos puede proteger ante desastres locales: si un centro de datos sufre un incidente, los datos duplicados en otros sitios aseguran la persistencia de la información y una rápida recuperación del servicio.

(Cabe mencionar que, junto con estas ventajas, las bases de datos distribuidas también pueden conllevar ahorros de costes en ciertos escenarios, al poder utilizar hardware común en clúster en vez de costosos mainframes centralizados. Sin embargo, esos ahorros se ven a veces contrarrestados por la mayor complejidad de administración, como se discutirá más adelante.)

Desafíos de la distribución de datos

Si bien distribuir los datos aporta beneficios importantes, **no está exento de desafíos**. Al pasar de un sistema centralizado a uno distribuido, surgen problemas complejos principalmente en torno a la **coordinación** de los nodos y la **sincronización** de la información entre ellos. Es fundamental entender estos retos, ya que son el contrapunto de las ventajas antes mencionadas:

- **Coordinación de múltiples nodos:** En un sistema distribuido, las diversas máquinas deben **trabajar en conjunto** para lograr los objetivos comunes (por ejemplo, responder a una consulta del usuario o ejecutar una transacción que afecta a datos en varios servidores). Lograr esta coordinación es mucho más **complejo** que en un sistema centralizado, pues ya no existe un solo punto de control secuencial. Los nodos tienen que comunicarse continuamente y **ponerse de acuerdo** sobre el estado de las operaciones. Esto implica implementar *protocolos* especiales para orquestar tareas distribuidas, asegurar que cada acción ocurra en el orden correcto y manejar casos donde alguna parte falle a mitad de proceso. Por ejemplo, si una transacción bancaria involucra cuentas en dos servidores distintos, ambos deben coordinarse para completar la operación o deshacerla en caso de error. Mecanismos como el protocolo de **compromiso de dos fases** (2PC) o algoritmos de consenso (como *Paxos* o *Raft*) se usan para garantizar acuerdos entre múltiples nodos, pero añaden sobrecarga y latencia. En general, “*la comunicación y la coordinación son más complejas*” en entornos distribuidos. Un error de coordinación puede llevar a inconsistencias o bloqueos del sistema, por lo que manejar

correctamente esta **orquestración distribuida** es uno de los grandes retos para los diseñadores de estos sistemas.

- **Sincronización y consistencia de datos:** Otro desafío crítico es mantener **consistentes** los datos replicados o particionados a través de distintos nodos. Cuando los datos se duplican en varios servidores (para lograr resiliencia o rendimiento), es necesario **sincronizar** las actualizaciones para que todos vean la misma información coherente. Garantizar esta *coherencia de datos* en diferentes ubicaciones puede resultar **difícil** debido a la latencia de la red y a posibles fallos de comunicación. Por ejemplo, si un cliente actualiza su información en la sede de Europa, ¿cómo y cuándo se propaga ese cambio al servidor de América? Podría haber un retraso durante el cual distintos usuarios ven datos diferentes. Los sistemas distribuidos deben utilizar protocolos de **replicación** (sincrónica o a veces *asíncrona*) y algoritmos de control de concurrencia distribuidos para resolver estos problemas. Sin embargo, tales mecanismos (como bloqueos distribuidos, timestamps lógicos, *quóruns* de lectura/escritura, etc.) complejizan mucho el diseño. Existe siempre un delicado equilibrio entre consistencia y rendimiento (descrito por el Teorema CAP, que indica que ningún sistema distribuido puede garantizar consistencia total, disponibilidad total y tolerancia a particiones de red simultáneamente). En la práctica, muchos sistemas optan por una **consistencia eventual**: los datos pueden divergir momentáneamente entre nodos, pero se *sincronizan* tras un breve lapso. Aun así, asegurar que *no haya conflictos ni pérdida de datos* requiere cuidadosas estrategias. En resumen, la *“necesidad de asegurar la sincronización y la coherencia de los datos entre diferentes sitios puede ser un desafío”* significativo en bases de datos distribuidas. Los administradores deben implementar soluciones robustas de replicación, monitoreo y reconciliación de datos para mantener la integridad del sistema.

En conclusión, **distribuir los datos** conlleva un *trade-off*: se obtienen rendimiento escalable y alta disponibilidad, a cambio de afrontar una mayor complejidad en el control del sistema. Las ventajas de **rendimiento, escalabilidad y resiliencia** hacen que las arquitecturas distribuidas sean esenciales en aplicaciones modernas de gran escala (desde servicios globales de internet hasta sistemas bancarios), permitiendo servir a millones de usuarios de forma eficiente. Pero para cosechar esos beneficios, es indispensable superar los desafíos de **coordinación y sincronización**, con arquitecturas y protocolos diseñados cuidadosamente. Solo así se garantiza que un sistema distribuido funcione de manera **confiable y consistente**, aprovechando al máximo el poder de la distribución de datos sin comprometer la calidad de la información ni la experiencia de los usuarios.

Replicación y Particionamiento (Sharding) en Bases de Datos Distribuidas

Replicación: Definición y Propósito

La **replicación** consiste en mantener copias idénticas de una base de datos en múltiples nodos o servidores. Es decir, se duplican los datos de forma que cada copia (llamada *réplica*) almacena la **misma** información que la base de datos original. El propósito principal de la replicación es lograr **tolerancia a fallos** y **alta disponibilidad**: si un nodo falla, otros tienen la misma información y pueden continuar dando servicio sin pérdida de datos. Adicionalmente, la replicación puede mejorar el rendimiento de ciertas cargas de trabajo, ya que las copias adicionales permiten **balancear consultas de lectura** en varios nodos (al haber varias réplicas para leer, se atienden más consultas en paralelo).

Analogía: *Imagina una biblioteca que hace copias idénticas de todos sus libros y las guarda en varias sucursales. Si una sucursal cierra, los lectores pueden acudir a otra y encontrar los mismos libros. Esto es replicación: duplicar la información para protegerse ante fallos y atender a más lectores simultáneamente.* Del mismo modo, en bases de datos, replicar es como tener “fotocopias” de los datos en sitios distintos para asegurar que siempre haya al menos una copia disponible.

A continuación, describimos los tipos de replicación más comunes y sus características técnicas, **ventajas**, **desafíos** y ejemplos.

Tipos de Replicación (Topologías)

Replicación Primaria-Secundaria (Master-Slave o Activo-Pasivo): En este esquema existe un nodo **primario** (master) que acepta todas las escrituras, y uno o varios nodos **secundarios** (replicas *slave* o pasivas) que reciben copias de los datos del primario. Los secundarios mantienen sincronizada la base de datos replicando los cambios (normalmente de forma continua y automática). Si la base de datos primaria falla, típicamente uno de los secundarios puede asumir el rol de primario mediante un proceso de **failover** (elección automática de nuevo primario), garantizando la continuidad del servicio.

- **Ventajas:** Sencillez en el orden de escrituras (solo un nodo escribe, evitando conflictos de escritura), y capacidad de **escala en lectura**: las consultas de solo lectura pueden repartirse entre el primario y los secundarios, mejorando el rendimiento cuando hay muchas lecturas concurrentes. También ofrece **alta disponibilidad** mediante la conmutación por falla – si cae el primario, un secundario actualizado toma su lugar automáticamente. Este modelo es fácil de entender y es común en sistemas de bases de datos relacionales tradicionales (por ejemplo, la replicación de MySQL/PostgreSQL en modo maestro-esclavo asíncrono, o los **conjuntos de réplicas** de MongoDB donde un nodo primario activo replica sus datos a secundarias).
- **Desafíos:** El primario puede convertirse en un **cuello de botella** para las escrituras, ya que toda operación de escritura debe pasar por él. La capacidad de escritura **no aumenta** añadiendo más réplicas (solo se escalan las lecturas). Además, puede haber un **retraso de replicación**: los secundarios podrían estar ligeramente desactualizados respecto al primario si la replicación es asíncrona, lo que implica lecturas “*obsoletas*” en secundarias (consistencia eventual).

También, durante el failover hay una breve ventana de reconfiguración en la que el sistema elige un nuevo primario, periodo en el cual las escrituras se detienen momentáneamente.

- **Ejemplos típicos:** La mayoría de las bases de datos relacionales ofrecen replicación primaria-secundaria (MySQL, PostgreSQL, SQL Server en modo espejo, etc.). MongoDB soporta este esquema mediante *replica sets* (un primario y N secundarios), al igual que Redis en su modo estándar líder-seguidor (*master-replica* en la terminología Redis). En estos sistemas, todas las escrituras van al líder/primario, y las lecturas pueden configurarse para salir de réplicas secundarias si se desea aliviar la carga del primario.

Replicación Multi-Master (Multi-primario o Activo-Activo): En este enfoque **no hay un único primario; varios nodos pueden aceptar escrituras simultáneamente**. Es decir, todos los nodos son *activos* y la información escrita en cualquiera de ellos se replica a los demás. Esta topología proporciona una configuración **activo-activo** donde las cargas de trabajo de escritura se distribuyen entre múltiples nodos.

- **Ventajas:** Permite **escalar las escrituras** y ofrece mayor disponibilidad de escritura, ya que **no existe un solo punto crítico** de fallo para las operaciones de actualización (si un nodo activo cae, otros siguen aceptando escrituras). También es útil en despliegues geográficamente distribuidos: por ejemplo, tener un nodo maestro en Europa y otro en América, de modo que los usuarios de cada región escriban en el nodo local reduciendo la latencia, replicándose luego los cambios entre regiones. Este esquema soporta mejor escenarios de múltiples centros de datos activos simultáneamente. Un ejemplo destacado es Apache **Cassandra**, que sigue un modelo inspirado en DynamoDB: todos sus nodos son equivalentes y cualquier nodo puede recibir escrituras; los datos se replican con un cierto *factor de réplica* sin que haya una réplica “maestra” central. En Cassandra, “todas las réplicas son igualmente importantes; no hay una réplica primaria o maestra”. Otros ejemplos incluyen algunos clústers de MySQL/MariaDB basados en Galera (que permiten múltiples maestros con replicación síncrona) y bases de datos *multi-master* como CouchDB o sistemas de sincronización como LDAP/Active Directory.
- **Desafíos:** El principal reto es la **consistencia**. Con múltiples maestros escribiendo a la vez, pueden surgir **conflictos de escritura** (por ejemplo, dos nodos actualizando el mismo registro de forma distinta al mismo tiempo). Resolver estos conflictos requiere estrategias adicionales (último valor en la línea de tiempo, fusiones lógicas de los datos, vectores de versión, etc.), aumentando la complejidad. Muchos sistemas multi-master optan por un modelo de **consistencia eventual**: los cambios se propagan de forma asíncrona y, en un periodo corto, todas las réplicas convergen al mismo estado, pero no se garantiza que las lecturas sean inmediatamente consistentes en todos los nodos. Esto significa que, sin mecanismos especiales, podríamos leer datos antiguos en un nodo justo después de una escritura reciente en otro. Además, **coordinar transacciones** con integridad fuerte es complejo en este esquema; lograr consistencia fuerte implica implementar algoritmos distribuidos de consenso (lo que añade latencia). En resumen, el desarrollo y mantenimiento

de una replicación multi-master robusta es más complicado que el esquema primario-secundario, por lo que no todas las bases de datos lo soportan de forma nativa.

Modos de Replicación: Síncrona vs Asíncrona

Independientemente de la topología (primario-secundario o multi-master), la replicación puede funcionar de forma **síncrona** o **asíncrona**, según *cuándo* se consideran confirmados los datos en las réplicas:

- Replicación Síncrona:** El nodo que recibe una escritura no la da por **confirmada** hasta asegurarse de que **al menos otra réplica** (o todas, según configuración) también ha escrito el dato con éxito en su almacenamiento. Es decir, la actualización se **aplica simultáneamente** al primario y a la réplica secundaria antes de reconocer la operación como completa. En términos prácticos, la aplicación que escribe debe esperar a que las réplicas remotas confirmen la escritura, logrando una replicación casi instantánea y totalmente consistente entre nodos. La **ventaja** de la replicación síncrona es que elimina la posibilidad de pérdida de datos en caso de falla repentina del nodo primario – siempre habrá al menos otra copia completamente actualizada. Sin embargo, su **desventaja** es el impacto en el **rendimiento y latencia**: al requerir confirmaciones remotas, las escrituras se vuelven más lentas, especialmente si las réplicas están geográficamente lejos o la red tiene alta latencia. Requiere además un ancho de banda de red robusto y estable, pues cualquier retraso o caída en la comunicación puede frenar las operaciones. En síntesis, la replicación síncrona garantiza máxima consistencia a costa de un menor throughput. *Ejemplos:* sistemas financieros críticos a veces emplean replicación síncrona para no perder ninguna transacción (p. ej., bases de datos Oracle Data Guard en modo síncrono, o MySQL Group Replication). También algunos clusters multi-master como Galera (Percona XtraDB Cluster) realizan una forma de replicación casi síncrona usando acuerdos de commit global. Otro ejemplo moderno es Google Spanner, que implementa consistencia fuerte distribuida usando consenso entre réplicas (efectivamente una replicación síncrona bajo el capó).
- Replicación Asíncrona:** En este modo, el nodo principal **no espera** confirmación inmediata de las réplicas. La escritura se considera *exitosa* una vez aplicada en el nodo origen, y luego se **propaga** a las réplicas en segundo plano o en intervalos de tiempo. En palabras simples, *“primero se escriben los datos en el nodo primario y luego se copian a la réplica”*. La mayoría de las implementaciones realistas funcionan así: el primario escribe los cambios en un registro de operaciones (*oplog* o binlog) y los secundarios los van leyendo y aplicando a su ritmo. La **ventaja** clara es el **alto rendimiento**: las escrituras regresan rápido al cliente sin esperar a otras máquinas, permitiendo mayor tasa de transacciones y tolerando mejor la latencia de red. Además, consume menos ancho de banda en tiempo real, ya que puede agrupar o retrasar un poco las transferencias de datos sin bloquear la operación principal. **El inconveniente** es la **ventana de inconsistencia temporal**: si el nodo principal falla de forma

catastrófica justo después de una escritura, existe riesgo de que esa última actualización **no haya llegado** a las réplicas, perdiéndose dicho dato. Por tanto, en caso de desastre podrían perderse los cambios más recientes. Asimismo, las réplicas asíncronas pueden quedarse ligeramente atrasadas (*lag*), lo que implica que lecturas desde un secundario podrían estar leyendo un estado antiguo. Pese a ello, la replicación asíncrona es la opción por defecto en la mayoría de sistemas debido a su eficiencia. *Ejemplos:* **MySQL** (replicación binlog tradicional) y **PostgreSQL** (streaming replication) trabajan de forma asíncrona por defecto; **MongoDB** replica sus conjuntos de réplicas de modo asíncrono (los secundarios aplican operaciones del *oplog* en cola); **Redis** también usa replicación asíncrona líder-seguidor por defecto para alta performance. En todos estos casos, suele haber opciones de semi-sincronía (por ejemplo, MySQL semisync plugin, comando WAIT de Redis, etc.) que permiten asegurarse de que al menos una réplica recibió los datos antes de confirmar, disminuyendo la probabilidad de pérdida de datos sin incurrir totalmente en la latencia de una síncrona estricta.

Particionamiento (Sharding): Definición y Distribución de Datos

El **particionamiento** (también conocido por el término en inglés *sharding*) es una técnica para **distribuir los datos de una base de datos en varios nodos**, pero a diferencia de la replicación, **dividiendo el conjunto de datos en partes diferentes** en lugar de duplicarlo completo en cada nodo. En concreto, la base de datos se fragmenta en **subconjuntos lógicos**, llamados *particiones* o *shards*, y cada partición contiene solo una porción de los datos totales. Cada partición suele residir en un nodo físico distinto (en sistemas distribuidos, “shard” a menudo implica un fragmento alojado en otro servidor). De esta manera, ninguna máquina almacena *todos* los datos, sino solo su trozo correspondiente.

El objetivo del particionamiento es lograr **escalabilidad horizontal** (*scale-out*): al repartir los datos entre múltiples servidores, se pueden manejar conjuntos de datos mucho más grandes que lo que cabría en una sola máquina, y también distribuir la carga de trabajo de manera que múltiples nodos atiendan peticiones en paralelo. En otras palabras, el sistema puede **manejar más volumen de información y más tráfico** sumando más nodos (shards), ya que cada uno trabaja con un subconjunto más pequeño. Esto mejora la capacidad de procesamiento global, especialmente para cargas intensivas en escritura o en almacenamiento.

Analogía: Siguiendo la metáfora de la biblioteca, imaginemos que en lugar de duplicar todos los libros en cada sucursal, decidimos **dividir el catálogo**: la sucursal A guarda libros de ciencias, la B libros de historia y la C libros de literatura. Ninguna sucursal tiene todo, sino una parte especializada. Esto es particionar la colección por tema. Si un lector busca un libro de literatura irá a la sucursal C. Así se reparte el servicio entre varias sedes. De forma similar, en una base distribuida particionada, cada nodo almacena solo ciertos registros (p. ej., clientes de Europa en un nodo y de América en otro, o rangos de IDs separados). El sistema en conjunto abarca todo el conjunto de

datos, pero **segmentado**; con ello se logra atender más solicitudes a la vez (cada nodo atiende las suyas) y almacenar más datos totales que en un solo servidor.

Otra forma de ver la diferencia: “*La fragmentación (sharding) particiona el conjunto de datos en partes discretas, mientras que la replicación lo **duplica***”. Particionar **no crea copias adicionales** de la misma información, sino que **divide** una base de datos en varias partes y las almacena en nodos distintos. Por sí solo, este enfoque **no aporta alta disponibilidad**, ya que si un nodo-partición se cae, los datos en esa partición quedan inaccesibles hasta que el nodo vuelva (a menos que combinemos con replicación, como veremos). El beneficio puro del sharding está en la **escalabilidad y paralelismo**: más nodos trabajando en partes diferentes implican más capacidad agregada.

Desafíos del particionamiento: implementar sharding agrega complejidad. Ya no existe una única base de datos central, sino múltiples fragmentos, lo que complica las operaciones. Por ejemplo, para responder ciertas consultas, puede ser necesario **consultar múltiples shards y combinar resultados**, aumentando la complejidad de las consultas y la latencia. También, la aplicación o el sistema de bases de datos debe ser capaz de **determinar en qué shard están los datos buscados** (normalmente mediante una clave de partición), de lo contrario habría que hacer *broadcast* de la consulta a todos los nodos. Además, puede ocurrir **desbalanceo**: si los datos no se distribuyen equitativamente, algunos shards pueden recibir mucho más tráfico o almacenar mucho más volumen que otros (fenómeno de *hot spots* o “partición caliente”). Elegir una buena estrategia de partición es crucial para evitar eso. El sistema debe considerar también la **reasignación** o *re-sharding* cuando se agregan más nodos: redistribuir datos entre más particiones puede ser costoso. Finalmente, mantener integridad referencial o transaccional a través de shards diferentes es complejo, ya que cada fragmento es independiente.

A pesar de estos retos, muchas bases de datos modernas (sobre todo **NoSQL** y **NewSQL**) han adoptado sharding como mecánica fundamental para el manejo de **Big Data**, debido a que el escalado vertical (hacer servidores más potentes) tiene límites físicos y costos crecientes. El *sharding* es una estrategia de escalamiento horizontal que, combinada con replicación, habilita arquitecturas capaces de manejar aplicaciones web masivas, redes sociales, etc., repartiendo datos y carga globalmente.

Ejemplos de Esquemas de Particionamiento por Clave

La forma en que se decide **qué datos van en cada partición** es mediante una **clave de partición** (shard key). A continuación, se presentan ejemplos típicos de criterios de sharding:

- **Por identificador de usuario (ID):** Un criterio común es distribuir según una identificación única del usuario o entidad principal. Por ejemplo, en un sistema multi-tenant, podríamos usar el *user_id* como clave de partición. Una técnica es aplicar una función *hash* al ID del usuario para asignarlo a un shard específico (obteniendo una distribución uniforme al azar). Así, todos los datos pertenecientes al usuario con ID 37 (su perfil, posts, etc.) residen en el shard X,

mientras el usuario 98 reside en el shard Y, etc. Esto asegura que la carga se reparte aproximadamente equitativa si hay muchos usuarios, y las consultas por usuario suelen ir a un único nodo. **Ventaja:** la cardinalidad de usuarios suele ser alta (muchos valores posibles) y uniforme, lo cual permite un buen reparto.

Desafío: requiere que las operaciones multi-usuario (p. ej., un join entre usuarios) se orquesten entre shards, pero en aplicaciones típicas las operaciones suelen estar *scopeadas* a un usuario a la vez, lo que hace esta partición efectiva.

- **Por región o país:** Aquí la clave de partición es geográfica. Por ejemplo, una empresa puede decidir que los datos de clientes **europeos** se almacenen en un shard (o centro de datos) en Europa, los de **América** en otro, Asia en otro, etc. Esto a veces es necesario por motivos de *latencia* (acercar físicamente los datos a los usuarios locales) o regulaciones (residencia de datos en su país). Un caso: *“un sitio de citas almacena usuarios de distintas ciudades en diferentes particiones físicas”*; las consultas locales de cada ciudad se atienden más rápido al estar en su partición local. **Ventaja:** las consultas regionales son eficientes y se puede optimizar la infraestructura para cada zona. **Desventaja:** la distribución de carga puede ser muy desigual si una región tiene muchos más usuarios que otra. Por ejemplo, usar **país** como shard key puede ser problemático porque países como EE.UU. o India pueden concentrar un volumen desmesurado de usuarios frente a otros. Esto llevaría a que el shard de “US” esté sobrecargado (*hot shard*) mientras otros están infrautilizados. En general, claves de baja cardinalidad (pocos valores distintos, como países ~200) tienden a producir distribución desequilibrada, a menos que la carga de trabajo se separe realmente por región.
- **Por rango de valores (fragmentación horizontal por rangos):** Consiste en asignar registros a shards según un rango continuo de la clave. Por ejemplo, particionar una tabla de clientes **alfabéticamente**: apellidos de A–H en el shard 1, I–Q en el shard 2, R–Z en el shard 3. O particionar facturas por año: cada año en un nodo distinto. **Ventaja:** es fácil determinar en qué shard está un dato por su valor, y agrupa datos similares juntos (útil para consultas por rangos, p. ej. rangos de fechas se resuelven en un shard). **Desafío:** si los datos no están uniformemente distribuidos en el dominio de la clave, habrá desequilibrio. Por ejemplo, si muchísimos apellidos comienzan con “A”, el shard de A–H recibirá más datos que otros. O si particionamos por rango de fechas y siempre llegan datos nuevos con fechas recientes, el shard más “nuevo” soportará todo el tráfico de inserción (esto pasó en algunos sistemas de logs particionados por fecha, causando un hotspot en la partición actual). El problema de **“cambio monótono”** se refiere a esto: si la clave crece de forma ordenada (monótona), los nuevos datos siempre caen en la misma partición al final del rango, volviéndola caliente. **Solución:** introducir hashing o límites que distribuyan mejor, o re-balanceo automático (como hace MongoDB con su *balancer* moviendo *chunks* entre shards para nivelar la carga).
- **Por hash de la clave (dispersión uniforme):** En lugar de usar directamente el valor de la clave (que podría tener patrones o rangos), se aplica una función **hash** a la clave y el resultado del hash determina el shard. Este método (partición hash) busca una **distribución uniforme** de los datos

independientemente de su significado. Por ejemplo, en Redis Cluster se usan 16,384 ranuras de hash para repartir las *keys* entre nodos; cada clave se mapea a una ranura hash y ésta a un nodo. **Ventaja:** logra balancear la carga de manera muy uniforme en promedio, evitando *hotspots* provocados por valores populares, ya que el hash aleatoriza la ubicación de cada ítem. **Desventaja:** rompe la *localidad semántica* de los datos – registros relacionados pueden terminar en nodos distintos si la clave difiere lo suficiente (p. ej., usuarios consecutivos podrían caer en shards distintos tras el hash). Además, agregar o quitar nodos requiere recalcular la distribución (aunque técnicas como *consistent hashing* mitigan cuánto se mueve). También dificulta ciertas consultas por rango, ya que el hash dispersa el orden natural.

- **Por directorio o categoría explícita:** Algunas particiones se basan en listas de valores categóricos predefinidos. Un ejemplo: dividir una base de datos de productos por **categoría de producto** (electrónica en un shard, ropa en otro, alimentos en otro). O en el caso de AWS mencionan *partición de directorios*: usar una tabla de búsqueda que asigne, por ejemplo, “color = azul” -> Partición A, “color = rojo” -> Partición B, etc.. **Ventaja:** es intuitivo y permite aislar conjuntos de datos conocidos (por ejemplo, separar clientes VIP en otro nodo). **Desafío:** requiere saber de antemano las categorías y puede sufrir desbalance si una categoría crece más que las otras. Además, si la asignación (directorio) contiene información errónea o desactualizada, los datos podrían ir al shard equivocado.

En la práctica, muchos sistemas combinan métodos: por ejemplo, primero particionar por región, y dentro de cada región por hash de ID, etc., para lograr tanto aislamiento geográfico como balanceo interno.

Diferencias entre Replicación y Particionamiento

Aunque ambos, replicación y sharding, son técnicas de bases de datos distribuidas, **sus objetivos y mecanismos son diferentes:**

- **Qué hacen con los datos:** La replicación **duplica** el conjunto de datos completo en varios nodos. En cambio, el particionamiento **divide** los datos en subconjuntos disjuntos. Con replicación, cada nodo tiene *lo mismo* (100% de los datos); con sharding, cada nodo tiene *una parte* y hay que unir las partes para obtener el 100%.
- **Meta principal:** La replicación se enfoca en la **alta disponibilidad y tolerancia a fallos**, y también en **mejorar la capacidad de lectura** (al tener múltiples copias para atender consultas). Si un nodo se cae, hay otro con los mismos datos listo para responder. En cambio, el particionamiento persigue la **escalabilidad y rendimiento** global del sistema, repartiendo la carga de datos y procesamiento (*sharding proporciona escalabilidad y paralelismo*). Permite manejar más usuarios, más transacciones y bases más grandes al sumar nodos que comparten el trabajo.
- **Tolerancia a fallos:** Un sistema solo con sharding **no** es intrínsecamente tolerante a fallos – cada fragmento es único. Si se cae el servidor de un shard,

esa porción de datos queda inaccesible, interrumpiendo parte del servicio (p. ej., los usuarios cuyos datos estaban en ese shard no podrán ser atendidos). Por el contrario, en la replicación, varias copias aseguran que **sigue habiendo disponibilidad** aunque falle un nodo. Por eso, *“a diferencia de la replicación, la fragmentación por sí sola no genera alta disponibilidad”*. Para mitigar esto, es común que **cada shard esté a su vez replicado**, así un fallo afecta a una máquina pero no a los datos de ese fragmento (ver más abajo sobre usos combinados).

- **Impacto en la escritura/lectura:** Con replicación (especialmente primaria-secundaria), las **escrituras** no ganan velocidad al añadir más réplicas (todas las escrituras siguen yendo al mismo punto), mientras que las **lecturas** sí pueden distribuirse entre copias. Con sharding, ocurre lo contrario: al dividir los datos, **cada escritura va solo al shard correspondiente**, de modo que añadir shards **sí aumenta** la capacidad total de escrituras soportadas (más nodos aceptando escrituras en paralelo, cada una para diferentes datos). Sin embargo, consultas de lectura complejas que necesiten datos de muchos shards podrían volverse más lentas si deben recopilar información de varios servidores; las lecturas sencillas por clave sí escalan linealmente con shards (cada nodo atiende las suyas).
- **Complejidad de aplicación:** La replicación suele ser **transparente** para la aplicación cliente: por ejemplo, en MongoDB o PostgreSQL, el driver/cluster se encarga de redirigir lecturas o de hacer failover en caso de caída de un nodo, sin que el desarrollador cambie la lógica de consulta. En cambio, el sharding **impacta en la lógica**: la aplicación o middleware debe saber, por la clave de búsqueda, a qué shard dirigir cada consulta, o el propio motor debe proveer un router (como el *mongos* en MongoDB). También, operaciones como *JOINS* o *transacciones* que involucren datos en distintos shards son muy complejas de implementar (requiriendo coordinación distribuida), a diferencia de replicación donde cada nodo tiene una copia completa y puede ejecutar cualquier JOIN localmente. En resumen, particionar exige mayor inteligencia en el enrutamiento de consultas y en mantener consistencia entre fragmentos, mientras que replicar exige mecanismos de sincronización de datos entre copias pero deja cada copia completa para consultar libremente.
- **Ejemplo numérico:** Si tienes 3 nodos y 75 GB de datos: con replicación **cada nodo** almacena los mismos 75 GB (3 copias de todo); con sharding, idealmente cada nodo almacena unos ~25 GB distintos (datos repartidos). En replicación gastas más espacio (duplicación) pero obtienes redundancia; en sharding aprovechas capacidad de almacenamiento sumada pero sin redundancia intrínseca.

En la práctica, **replicación y particionamiento no se excluyen**, de hecho **se complementan** para lograr sistemas escalables y robustos. Una frase que resume su papel combinado es: *“Sharding proporciona escalabilidad (más datos, más throughput) mientras que la replicación proporciona disponibilidad (tolerancia a fallos)”*. Veamos a continuación cómo se usan conjuntamente.

Uso Combinado de Replicación y Sharding

En sistemas distribuidos a gran escala es común **usar ambos enfoques simultáneamente: particionar los datos** entre muchos nodos y, a la vez, **replicar cada partición** en nodos redundantes. De este modo se obtiene lo mejor de ambos mundos: **escalabilidad horizontal y alta disponibilidad**.

Por ejemplo, MongoDB en despliegues de producción recomienda esta combinación: se crea un **cluster sharded** donde **cada shard es en realidad un conjunto de réplicas**. Así, imaginemos un cluster con 3 shards; cada shard contiene una porción distinta de los datos, y para que cada porción sea redundante, definimos cada shard como un replicaset de, digamos, 2 nodos (1 primario y 1 secundario). En total tendríamos 3 shards $\times$ 2 nodos = 6 nodos. Si un nodo dentro del shard falla, su réplica gemela se hace cargo (no se pierde ese fragmento); y si todo un shard se sobrecarga, podríamos agregar un nuevo shard (que a su vez tendría sus réplicas) para repartir los datos. En la pregunta de un estudiante citado en un foro, se lo aclaraba así: *“Usar ambos significa que estarás replicando shards; en otras palabras, **cada shard (fragmento) es a su vez un grupo de réplicas**”*. De este modo, un cluster con N shards y R réplicas por shard tendrá $N \times R$ nodos en total, por ejemplo un cluster de “3 shards replicados por 3” tendría 9 nodos.

Ilustración MongoDB: En un clúster de MongoDB, el *sharding* y la replicación trabajan juntos para lograr tanto escalabilidad como disponibilidad. El **sharding** divide los grandes conjuntos de datos en trozos más pequeños y los distribuye entre múltiples shards, permitiendo escalar horizontalmente a medida que los datos crecen. **Mientras tanto, cada shard se configura como un conjunto de réplicas** — con un primario y uno o más secundarios — garantizando que los datos de ese shard se mantengan altamente disponibles y tolerantes a fallos. *“Si un primario falla, un secundario toma el mando transparentemente, manteniendo el sistema resistente”*. Esta combinación implica que el sharding reparte la carga y mejora el rendimiento distribuyendo los datos, **y la replicación salvaguarda la información** al duplicarla, asegurando redundancia. En la literatura se afirma precisamente: *“Mientras el sharding maneja la carga e incrementa el rendimiento al distribuir datos entre shards, la replicación resguarda la información con copias redundantes”*.

Otros ejemplos de uso combinado incluyen a **Redis Cluster**, que realiza *sharding* del espacio de claves (usando hashing a 16384 slots) para repartir datos entre varios nodos maestros, y a la vez cada nodo maestro puede tener réplicas esclavas para failover. En **Cassandra**, aunque el concepto de shard no es explícito, internamente usa *particionamiento* (por hash de la clave) para distribuir datos en su anillo, y luego almacena múltiples réplicas de cada partición en diferentes nodos según el *replication factor* configurado. Es decir, Cassandra combina nativamente ambas ideas: cada porción de datos se **replica en varios nodos** para tolerancia a fallos, y ninguna máquina tiene toda la data (solo su porción asignada más copias de porciones de otros).

¿Cuándo usar ambos? En casi cualquier caso de **sistemas de datos de gran escala** se termina usando una combinación. Si una aplicación requiere tanto manejar **volúmenes masivos** de información (o altísimas tasas de transacciones) y debe estar **siempre disponible**, la arquitectura típica es: **shardear para escalar** (añadir nodos ante

crecimiento) + **replicar para sobrevivir fallos** (redundancia). Por ejemplo, una red social con miles de millones de usuarios: se particiona por usuario o región para distribuir la base de datos en muchos servidores, y cada servidor con réplicas para no perder datos ni servicio ante caídas. Otro caso es cuando se necesita despliegue global: se pueden combinar partición por región (para latencia) y replicación dentro de cada región (para disponibilidad local), e incluso replicar ciertos shards entre regiones para backups o analytics.

En resumen, replicación y particionamiento son **herramientas complementarias** en el diseño de bases de datos distribuidas. Cada una aporta algo diferente (redundancia vs. división de carga). La replicación **garantiza disponibilidad y seguridad de los datos** ante fallos, mientras que el particionamiento **permite crecer en capacidad** de forma casi ilimitada agregando más nodos. No son excluyentes sino que, en las arquitecturas modernas, suelen **usarse en conjunto**: *“La partición se puede usar en combinación con la replicación para lograr escalabilidad y alta disponibilidad”*. Así, se alcanzan sistemas robustos, capaces de manejar grandes volúmenes de información de manera eficiente y resiliente, que es finalmente el objetivo de las bases de datos distribuidas en entornos de ingeniería actuales.

MODELOS DE CONSISTENCIA EN SISTEMAS DISTRIBUIDOS

En los sistemas distribuidos (con datos replicados en múltiples nodos), el **modelo de consistencia** define qué tan **sincronizados y coherentes** estarán los datos vistos por los clientes en diferentes circunstancias. A continuación analizaremos distintos modelos de consistencia —desde el más estricto al más laxo— explicando sus definiciones, analogías intuitivas, ventajas, desventajas y ejemplos tanto cotidianos como de tecnologías reales. Finalmente, discutiremos la relación entre consistencia y disponibilidad, un compromiso fundamental en la ingeniería de sistemas distribuidos.

Consistencia fuerte (Strong Consistency)

Definición técnica: La consistencia fuerte (también llamada *linealizabilidad* o consistencia atómica) garantiza que **todas las lecturas vean siempre el dato más reciente** que se haya escrito, como si hubiera una única copia actualizada de los datos en todo el sistema. En términos formales, es una garantía de *actualidad*: el sistema se comporta como si solo existiera una copia única, donde cada operación de lectura/escritura es atómica y globalmente ordenada en tiempo real. Esto significa que una vez confirmada una escritura, cualquier lectura posterior (desde cualquier nodo) obtendrá ese valor actualizado más reciente.

Explicación accesible: Podemos imaginar la consistencia fuerte como si **todas las máquinas compartieran un mismo libro de registros**. Cuando alguien escribe un nuevo dato en el libro, *instantáneamente* todos los lectores ven ese cambio porque siguen leyendo del mismo libro central. Es como un *registro maestro único*: por ejemplo, si transfieres dinero en una cuenta bancaria, *cualquier cajero o app móvil* que consulte el

saldo inmediatamente después verá la actualización. No importa a qué sucursal o réplica preguntes, siempre obtendrás la información más reciente, dando la **ilusión de una sola fuente de verdad** en tiempo real. Para los desarrolladores, esto simplifica el razonamiento ya que el comportamiento equivale al de un sistema no distribuido – evita sorpresas de datos desactualizados.

Ventajas: *Strong consistency* ofrece importantes beneficios para ciertos escenarios:

- Garantiza **correctitud absoluta**, evitando datos obsoletos y anomalías graves (por ejemplo, elimina lecturas *stale* y previene situaciones como doble gasto de dinero).
- Simplifica la lógica de las aplicaciones al brindar una vista coherente única de los datos en todo momento. Los clientes siempre ven un estado consistente, facilitando asumir invariantes (útil en finanzas, control de inventarios estrictos, etc.).
- Mantiene un **orden global** de operaciones fácil de razonar. Esto es crucial en casos como transacciones bancarias, sistemas de reservas (no vender dos veces el mismo asiento) o elección de líderes en clústeres, donde todos los nodos deben acordar exactamente el mismo estado actualizado.

Desventajas: Estas garantías fuertes vienen con costos significativos:

- **Mayor latencia:** para asegurar que todas las réplicas reconozcan cada operación en orden, a menudo se requieren protocolos de consenso (quóruns, Paxos/Raft, etc.) que añaden demoras. Las escrituras deben confirmarse en múltiples nodos, lo que aumenta el tiempo de respuesta.
- **Menor disponibilidad ante fallos:** si algunas réplicas/nodos fallan o hay una partición de red, un sistema de consistencia fuerte puede **bloquear las operaciones** (dejando de responder) hasta restablecer la sincronización completa. En otras palabras, para no arriesgar inconsistencia, puede sacrificar disponibilidad: las escrituras (y a veces lecturas) se rechazan mientras no se tenga confirmación de todos los nodos necesarios.
- **Costo y complejidad:** mantener consistencia estricta en sistemas geográficamente distribuidos requiere mucha coordinación. Protocolos como *two-phase commit* o replicación sincrónica encarecen las operaciones. Escalar con consistencia fuerte es desafiante, y a menudo hay un *trade-off* directo entre rendimiento y este nivel de consistencia.

Ejemplos concretos:

- *Situación cotidiana:* Un **cajero automático bancario** o sistema de pagos en tiempo real aplica consistencia fuerte. Si acabas de depositar dinero en una cuenta, cualquier otro cajero consultado segundos después reflejará inmediatamente el nuevo balance (no existe una “ventana” donde otro cajero muestre el saldo antiguo). Esto es vital para prevenir errores como gastar dinero que ya se gastó desde otro terminal.

- **Tecnologías reales:** Muchas bases de datos relacionales tradicionales en configuración centralizada brindan esta consistencia (por ejemplo, una base de datos SQL con transacciones ACID en un solo nodo). En entornos distribuidos, **Google Spanner** es un ejemplo destacado: es una base de datos globalmente distribuida que ofrece *consistencia transaccional fuerte*, de modo que “*cada lectura refleja las actualizaciones más recientes*” sin importar la escala o ubicación de los datos. Servicios como **Zookeeper** o **etcd** (usados para coordinación distribuida) también implementan consistencia fuerte (modelo CP) para asegurarse de que los clientes vean un estado único ordenado – por ejemplo, al usar Zookeeper para electores de líder, todos los nodos acuerdan exactamente quién es el líder actual en todo momento.

Lectura después de escritura (Read-Your-Writes)

Definición técnica: La consistencia *read-your-writes* (lectura de tus propias escrituras) es una garantía *centrada en el cliente* que asegura que **después de que un cliente realice una escritura, cualquier lectura subsecuente hecha por ese mismo cliente incluirá ese cambio**. En términos simples: si tú escribes (actualizas) un dato X, entonces tus lecturas posteriores de X no verán la versión antigua sino que *siempre* verán al menos lo que acabas de escribir (o una versión más nueva). Es una propiedad a nivel de sesión/usuario, más débil que la consistencia fuerte global, pero que **evita una anomalía frustrante: escribir algo y no verlo reflejado en tu siguiente lectura**.

Explicación accesible: Como analogía, imagina que **escribes una nota en una pizarra** y luego la vuelves a mirar – naturalmente esperas ver tu nota ahí. En un sistema distribuido sin esta garantía, podría ocurrir que escribes un dato en un servidor, luego consultes rápidamente otro servidor y ¡tu cambio aún no aparezca! (*Read-your-writes* evita esta sorpresa.) Un ejemplo común es cuando publicas una foto o estado en una red social: esperas que, al actualizar tu feed inmediatamente después, *tu propia publicación aparezca allí*. Con consistencia eventual pura podría fallar (tal vez tu publicación aún no se ha replicado a todos los servidores y tú podrías ser direccionado a uno desactualizado). Pero con una garantía de lectura tras escritura, el sistema se asegura de que **siempre veas tus propios cambios de inmediato**, mejorando la experiencia del usuario al dar coherencia con sus acciones recientes.

Ventajas: Esta modalidad aporta:

- **Experiencia de usuario coherente:** El usuario nunca se confunde por “cambios fantasmas” que desaparecen. Cada vez que realiza una actualización y luego consulta, verá al menos lo que escribió. Esto es crucial en aplicaciones interactivas (redes sociales, blogs, editores colaborativos) para que las acciones del usuario tengan efecto visible inmediato.
- **Implementación relativamente simple por sesión:** En muchos casos, *read-your-writes* se puede lograr dirigiendo siempre las lecturas del usuario a la réplica donde realizó la escritura, o usando un token de sesión que marca hasta qué actualización ha visto el cliente. No requiere consenso global, solo asegurar consistencia **para cada cliente individual**.

- **Evita pérdida de datos desde la perspectiva del cliente:** Desde la óptica de quien escribe, garantiza que sus datos no se “pierden en el camino” – brinda confianza en que el sistema registró sus cambios.

Desventajas:

- **No garantiza consistencia global:** Importante destacar que *read-your-writes* es una condición local al cliente. Otros usuarios podrían no ver inmediatamente la escritura (a menos que el sistema también ofrezca otras garantías). El sistema en conjunto podría seguir siendo eventualmente consistente globalmente; esta es solo una mejora de *sesión*.
- **Complejidad de sesión/caché:** Implementar esta garantía puede requerir mecanismos adicionales: por ejemplo, mantener *sesiones pegajosas* (sticky sessions) para que un usuario consulte siempre el mismo nodo tras escribir, o propagar su escritura a cualquier nodo del que haga lecturas. También se puede lograr almacenando en caché local del cliente los valores escritos. Todo esto añade cierta complejidad en la capa de cliente o de enrutamiento.
- **Coste en replicación acelerada:** Si se desea que *read-your-writes* se cumpla incluso cuando el siguiente read llega a una réplica diferente, el sistema podría forzar una sincronización más rápida o consultas con token de versión. Esto podría impactar ligeramente el rendimiento o la simplicidad de un sistema puramente eventual (aunque sigue siendo menos costoso que exigir consistencia fuerte global).

Ejemplos concretos:

- *Situación cotidiana:* Piensa en un **foro o plataforma de comentarios**. Cuando publicas un comentario, la página usualmente te lo muestra de inmediato junto al hilo, incluso si otros usuarios tal vez aún no lo ven hasta refrescar. Esa “ilusión” se logra porque para ti el sistema asegura lectura tras tu escritura. Otro ejemplo: en servicios de **almacenamiento en la nube**, cuando subes un archivo y luego lo buscas en tu lista, esperas verlo listado inmediatamente. (De hecho, Amazon S3 hoy garantiza consistencia de lectura-después-de-escritura para nuevos objetos: si subes un archivo nuevo, cualquier lectura siguiente lo verá inmediatamente).
- *Tecnologías reales:* **Amazon S3** (almacenamiento de objetos) históricamente era eventual, pero ahora *para objetos nuevos* provee *strong read-after-write*: una vez subido un nuevo objeto, cualquier GET posterior lo obtendrá sin demora. En cambio, si modificas o borras un objeto existente en S3, aplica consistencia eventual (un GET inmediato podría retornar la versión anterior debido a replicación retrasada). Otro ejemplo es **Azure Cosmos DB** en nivel de consistencia *Session*: es el nivel predeterminado que garantiza *read-your-writes* para cada sesión de cliente. Muchas bases NoSQL con replicación asíncrona (MongoDB, Cassandra, DynamoDB, etc.) no son fuertemente consistentes por defecto, pero suelen implementar esta garantía de sesión. Por ejemplo, en **DynamoDB** podrías leer por defecto datos eventualmente consistentes; sin embargo, al usar *condiciones de lectura transaccional* o mantener la sesión,

puedes asegurarte de leer tus escrituras previas. En general, cualquier *caché local* en aplicaciones web que almacena los cambios del usuario hasta que el servidor los confirme es una forma de ofrecer *read-your-writes* percepción al usuario.

Consistencia eventual (Eventually Consistent)

Definición técnica: La consistencia eventual es un modelo de consistencia *débil* donde el sistema no garantiza que todas las réplicas coincidan en todo momento, pero sí asegura que **si no se realizan nuevas actualizaciones, eventualmente todas las copias convergerán al mismo valor**. En otras palabras, tras un periodo de propagación (no especificado), *todas* las réplicas terminarán viendo la última escritura conocida. Durante el ínterin, pueden existir discrepancias: un cliente podría leer datos obsoletos en alguna réplica. Eventual es el modelo más relajado comúnmente utilizado en sistemas tolerantes a fallos: prioriza disponibilidad y rendimiento sobre la inmediatez de la consistencia.

Explicación accesible: Una analogía clásica es **el chisme que se esparce**: cuando alguien difunde una noticia en un grupo grande, no todos la saben al instante, pero *con el tiempo* cada miembro terminará escuchándola. Puede haber momentos en que algunos tengan información vieja y otros ya la actualizada, pero finalmente todos se pondrán al día. En un sistema eventualmente consistente, las actualizaciones a los datos se replican de forma asíncrona en segundo plano (como *gossip* entre nodos) y mientras tanto diferentes usuarios podrían obtener respuestas distintas según a qué réplica preguntan. Por ejemplo, si actualizas tu perfil en una red social, quizás un amigo lo ve de inmediato desde un servidor cercano actualizado, pero otro amigo, conectado a otra réplica aún no sincronizada, podría ver la versión antigua por unos instantes. **No hay una ordenación global instantánea**, pero se confía en la *convergencia*: eventualmente todos los nodos reflejarán el mismo estado actualizado.

Ventajas:

- **Alta disponibilidad y resiliencia:** Como no se espera a sincronizar con todas las réplicas para cada operación, el sistema puede seguir atendiendo solicitudes incluso si hay nodos caídos o desconectados. Esto lo hace ideal para sistemas que deben seguir funcionando ante fallos de red. De hecho, en condiciones de partición de red, un sistema de consistencia eventual puede permitir que cada partición opere independientemente (continuando disponible) aceptando que divergirán temporalmente.
- **Baja latencia y rendimiento óptimo:** Las escrituras y lecturas pueden completarse rápidamente en la réplica local sin bloqueo global. Al replicar asíncronamente, los usuarios obtienen respuestas más rápidas. Se pueden lograr tasas de *throughput* mayores, ya que no se requiere coordinación en cada operación. Por ejemplo, **DynamoDB** por defecto usa lecturas eventualmente consistentes, lo cual maximiza el rendimiento y minimiza costos – se pueden aprovisionar menos recursos si la aplicación tolera datos potencialmente desactualizados.

- **Escalabilidad sencilla:** Este modelo se adapta bien a escalas masivas (redes de distribución de contenido, caches globales, etc.), ya que cada nodo atiende peticiones con sus datos locales y sincroniza en segundo plano. No hay un cuello de botella central por consenso. Permite replicación en geografías distantes sin incurrir en tiempos de espera largos por la latencia de red en cada operación. Muchos sistemas geo-distribuidos optan por eventual consistency para brindar experiencias *locales rápidas* en distintas regiones.

Desventajas:

- **Lecturas de datos obsoletos:** La principal desventaja es obvia: un cliente puede leer información antigua (*stale*) sin saberlo. El sistema no promete devolver el último valor escrito en una lectura, lo que puede ser problemático para ciertas aplicaciones. Por ejemplo, un usuario podría ver un inventario incorrecto (p. ej., todavía muestra stock disponible que ya se agotó en otra región hace segundos).
- **Complejidad para desarrolladores:** Programar aplicaciones sobre consistencia eventual es más complejo, ya que hay que manejar explícitamente la posibilidad de inconsistencias. La lógica de la aplicación debe tolerar valores cambiantes o reconciliar actualizaciones en conflicto. Por ejemplo, en **Amazon Dynamo** (y su descendiente Cassandra) se aplican técnicas de *resolución de conflictos* como *last-write-wins* o *versionado con vectores* para cuando dos réplicas reciben escrituras diferentes durante la desconexión. El programador debe considerar estos casos (p. ej., dos usuarios modifican el mismo dato en distintas réplicas aisladas – al reconectar, ¿cómo se fusiona?).
- **Orden de lectura no garantizado:** Sin garantías adicionales, pueden ocurrir efectos contraintuitivos para los usuarios. Un sistema meramente eventual *no asegura* propiedades como *read-your-writes* ni *lecturas monótonas*. De hecho, **un sistema estrictamente eventual puede exhibir anomalías:** por ejemplo, un usuario podría leer un dato que él mismo actualizó y ver la versión vieja (falta de lectura de tus escrituras), o leer valores que retroceden en el tiempo al hacer varias lecturas seguidas (falta de monotonidad). Estas inconsistencias *por sesión* requieren añadir garantías extra para evitarlas (ver más adelante las lecturas/escrituras monótonas).
- **Eventual puede ser demasiado débil para ciertas transacciones:** Si necesitas invariantes fuertes (como “nunca sobregirar una cuenta bancaria” o “un usuario único por nombre”), eventual consistency por sí sola no las asegura sin mecanismos complementarios. Por ello, muchos sistemas eventualmente consistentes limitan su uso a *ciertos dominios* donde una ligera divergencia temporal es aceptable.

Ejemplos concretos:

- **Situaciones cotidianas:** Un ejemplo común es el **Sistema de Nombres de Dominio (DNS)** en Internet. Cuando actualizas la dirección IP de un dominio, ese cambio no se refleja inmediatamente en todos los servidores DNS del mundo; en su lugar, se propaga gradualmente. Durante ese lapso, distintas

personas pueden obtener respuestas distintas (unos resuelven la IP nueva, otros la antigua), pero al cabo de cierto tiempo (TTL) todos los caches convergerán al nuevo valor. DNS es deliberadamente eventualmente consistente, priorizando que el servicio siga resolviendo dominios incluso si algunas actualizaciones aún no llegan a todos. Otro ejemplo: **redes sociales** como Twitter o Facebook históricamente manejan los contadores de “me gusta” o “retweets” de forma eventual. Es decir, cuando das *like* a un post, podrías ver el contador incrementado localmente, pero otros podrían no ver ese incremento instantáneamente hasta que sus aplicaciones reciban la actualización. Mientras tanto, el sistema sigue 100% disponible y rápido, y *eventualmente* todos verán el mismo número actualizado.

- **Tecnologías reales:** Muchas bases de datos NoSQL distribuidas implementan consistencia eventual por omisión. **Apache Cassandra**, por ejemplo, usa replicación asíncrona entre nodos y considera que “todas las réplicas llegarán a sincronizarse eventualmente”. Permite lecturas muy rápidas, aunque potencialmente desactualizadas, a menos que el cliente explícitamente pida cierto nivel de consistencia (por ejemplo, lectura con *quorum*). **Amazon DynamoDB** ofrece la opción de lecturas eventualmente consistentes (más rápidas y baratas) frente a lecturas fuertemente consistentes (más lentas). Servicios de alta escalabilidad como **Amazon SimpleDB** o antiguamente **CouchDB** también están diseñados bajo el principio de eventual consistency. Otra instancia son las **cachés distribuidas** y **redes CDN**: generalmente, cuando se actualiza un contenido, no se invalida instantáneamente en todos los nodos del edge, sino que cada nodo va refrescando conforme expira el contenido, ofreciendo consistencia eventual pero asegurando que los usuarios siempre obtengan *alguna* respuesta (aunque pueda ser una versión previa del contenido) incluso durante fallos de enlace.

Lecturas y escrituras monótonas (Monotonic Reads/Writes)

Estos modelos definen garantías de orden para operaciones sucesivas de un mismo cliente, mejorando la experiencia de consistencia sin llegar a ser fuerte. Veamos cada una:

Lecturas monótonas (Monotonic Reads): Un sistema ofrece *consistencia de lectura monótona* si **cuando un proceso (o usuario) ha leído un valor de un dato, cualquier lectura posterior de ese mismo dato por el mismo proceso devolverá ese valor o uno más reciente, pero nunca retrocederá a un valor más antiguo**. Dicho de otra forma, las lecturas de un cliente son *no decrecientes en el tiempo*: su visión de los datos solo puede estancarse o avanzar, pero no ir hacia atrás.

- **Ejemplo intuitivo:** Imagina que revisas tu **correo electrónico** en tu laptop por la mañana y ves un mensaje nuevo. Más tarde, desde tu teléfono, abres nuevamente el correo: esperas ver *al menos* ese mensaje (y posiblemente más nuevos), pero sería muy extraño que no apareciera (es decir, que la vista del teléfono estuviera “atrasada” respecto a lo que ya viste antes). La lectura monótona garantiza exactamente eso: una vez has visto cierto email, no

“olvidarás” su existencia en lecturas posteriores desde otros servidores. Los sistemas de email distribuido suelen implementarlo: cada servidor de correo al que te conectas se asegura de tener todos los emails que ya viste, sincronizando si es necesario, para no presentarte una bandeja incompleta.

- **Ventaja:** Evita la confusión de información que **va hacia atrás en el tiempo**. El usuario disfruta de una experiencia consistente: nunca leerá datos antiguos después de haber visto datos más nuevos. Esto mejora la confianza en el sistema. Para el desarrollador, garantiza que no necesita manejar escenarios en que, por ejemplo, un registro aparente “desaparezca” para un usuario temporalmente.
- **Desventaja:** Requiere coordinar o recordar el *último timestamp/version visto* por cada cliente. Implementaciones típicas pueden involucrar que el cliente siempre consulte la misma réplica líder, o que las réplicas se sincronicen hasta al menos el punto leído antes de responder al cliente. Esto añade cierta latencia en algunos casos. Si un cliente rota entre servidores, esos servidores deben comunicarse para actualizarse hasta el punto más reciente visto por el cliente, introduciendo overhead. Además, lectura monótona por sí sola no asegura que el cliente vea *sus propias escrituras* (esa es la propiedad read-your-writes), ni resuelve inconsistencias entre diferentes clientes; es solo una garantía local de orden temporal de lecturas.

Escrituras monótonas (Monotonic Writes): Un sistema es *monotonic-write consistent* si **un cliente que realiza múltiples escrituras lo hace en orden, y el sistema garantizará que esas escrituras se aplican en el mismo orden emitido**. En concreto, si una misma persona hace primero la actualización A y luego la B sobre un dato X, no habrá forma de que B se haga visible antes que A; *A siempre precederá a B* en la vista de cualquier observador. Este modelo evita que escrituras más nuevas sean “pisadas” o ocultas por escrituras anteriores del mismo emisor.

- **Ejemplo intuitivo:** Piensa en publicar en una **red social** dos actualizaciones consecutivas: primero “Hola mundo”, luego “Cómo están todos?”. Con escrituras monótonas, tus seguidores nunca verán el segundo mensaje antes que el primero; el orden de publicación se preserva para todos. Si el sistema no lo garantizara, un escenario problemático sería que debido a replicación asíncrona, algunos usuarios vieran el segundo post pero *no* el primero (porque llegó a distintas réplicas en distinto orden), causando incoherencia narrativa. Monotonic writes evita esas inversiones. Otro ejemplo: en un **sistema de archivos replicado**, si guardas la versión 1 de un documento y luego la versión 2, todos los servidores deben aplicar primero la v1 y luego la v2; ninguno debería registrar la v2 y posteriormente “dar un paso atrás” a la v1.
- **Ventaja:** Asegura la **integridad de la secuencia de actualizaciones de cada cliente**, previniendo anomalías donde una actualización más nueva se pierde detrás de una vieja. Esto mantiene la lógica de la aplicación consistente. Por ejemplo, en un sistema de control de versiones distribuido, la garantía de escrituras monótonas impide que una versión antigua sobrescriba a una nueva accidentalmente. También facilita razonamiento: si un usuario hizo cambio A

antes que B, cualquier persona u nodo verá A antes que B, respetando causalidad básica entre las acciones de ese usuario.

- **Desventaja:** Para lograrlo, a menudo se necesita *ordenar las escrituras por cliente*, lo que implica algún mecanismo como colas por origen, timestamps lógicos, o forzar que un mismo nodo coordine las escritas de ese cliente. En sistemas totalmente multi-maestro sin coordinación, esta propiedad puede romperse (un cliente podría conectar a dos réplicas distintas en poco intervalo y sus escrituras llegar desordenadas). Garantizar escrituras monótonas puede reducir algo la **conurrencia** (p.ej., el cliente debe esperar confirmación de que su escritura anterior se replicó antes de emitir la siguiente). También, esta garantía es local a cada cliente y no evita conflictos entre diferentes clientes concurrentes.

Ejemplos concretos:

- *Situación cotidiana:* Un **blog colaborativo** con múltiples autores podría beneficiarse de ambas garantías. Por ejemplo, si tú (como autor) publicas primero el capítulo 1 y luego el capítulo 2, los lectores no deberían ver el capítulo 2 aparecer sin que esté el 1 (escrituras monótonas mantienen el orden). Y si un lector ya vio hasta el capítulo 2 ayer, hoy no debería ver solo el capítulo 1 porque visitó otro mirror del sitio (lectura monótona). En la práctica, sistemas de replicación web y CDN suelen implementar algo de esto asegurando orden de publicación por autor y sirviendo contenido no retroactivo por usuario mediante cachés con invalidación apropiada.
- *Tecnologías reales:* **Azure Cosmos DB (Consistencia de Sesión)** garantiza tanto lecturas monótonas como escrituras monótonas y lectura de tus escrituras dentro de una sesión de usuario. Es decir, el *nivel de consistencia de sesión* implementa todas estas garantías client-centric: el cliente no verá retrocesos en lecturas ni sus escrituras aparecerán fuera de orden. Otra tecnología, **Causal Consistency** (implementada en algunas bases de datos y tiendas como Redis Causal Clustering o ciertos modos de MongoDB), abarca escrituras monótonas junto con otras propiedades de causalidad; asegura que las escrituras de cada nodo se propagan en orden FIFO. Por ejemplo, **Bayou** (un sistema de base de datos replicada temprana de los '90) introdujo estas *garantías de sesión* para mejorar usabilidad en sistemas eventualmente consistentes. Finalmente, **sistemas de archivos distribuidos** como Coda o Git en modo distribuido mantienen orden de escrituras por repositorio/cliente mediante números de secuencia, de modo que las operaciones se aplican en el orden emitido, logrando escrituras monótonas. En general, estos modelos suelen ser invisibles para el usuario final pero implementados en la capa de protocolo para evitar anomalías temporales en la replicación.

Disponibilidad vs. consistencia en sistemas distribuidos



Figura – Trade-off entre consistencia y disponibilidad/latencia: los modelos van desde **fuerte** (consistencia máxima, pero menor disponibilidad y mayor latencia) hasta **eventual** (alta disponibilidad y rapidez, a costa de datos potencialmente desactualizados). La elección del modelo determina qué se sacrifica en situaciones de fallo de red.

Un concepto fundamental en sistemas distribuidos es que **no se puede tener al mismo tiempo 100% de consistencia y 100% de disponibilidad bajo particiones de red**. Esto se conoce comúnmente como el compromiso **CAP** (Consistencia, Disponibilidad y Tolerancia a Partición). Intuitivamente: **¿qué sucede si una red se divide (partición) o un nodo se cae?** Tenemos dos opciones:

- **Priorizar consistencia:** detener o limitar las operaciones hasta que los nodos puedan comunicarse de nuevo para sincronizar datos. Esto garantiza que el estado permanezca consistente en todas las réplicas, *pero el sistema deja de estar disponible* para algunas operaciones durante la caída. En otras palabras, el sistema elige estar “en lo correcto pero inactivo”.
- **Priorizar disponibilidad:** permitir que cada partición aislada o nodo siga operando y respondiendo a clientes con la información que tenga, incluso si está potencialmente desactualizada respecto a otras partes. Así el sistema continúa funcionando (alta disponibilidad), *pero los datos pueden divergir temporalmente*, rompiendo la consistencia global. Es decir, el sistema opta por estar “activo pero posiblemente equivocado”.

Cuando ocurre una partición de red real, un sistema distribuido *debe* sacrificar uno de esos dos atributos. **Si decidimos que el sistema siga respondiendo pase lo que pase, entonces no podemos garantizar que las respuestas sean las más recientes (consistentes)**. Por otro lado, **si insistimos en respuestas perfectamente consistentes, entonces ante una partición el sistema tendrá que dejar de responder (no disponible) mientras espera la recuperación de la comunicación**. No hay magia que permita ambas cosas simultáneamente en ese escenario.

Preferir disponibilidad vs preferir consistencia en la práctica: En diseño de sistemas, esto se traduce en elegir entre arquitecturas tipo **AP** (Available/Partition-tolerant) o **CP** (Consistent/Partition-tolerant) en el contexto CAP. *Preferir disponibilidad* significa que en caso de fallo, el sistema **sigue aceptando operaciones**. Por ejemplo, muchos servicios web de alto tráfico eligen esta ruta: es mejor dar al usuario *alguna respuesta* aunque pueda estar basada en datos viejos, que no dar ninguna. Un caso típico es una

tienda en línea durante un evento de alto tráfico: quizás los servidores de distintas regiones pierden conectividad entre sí; un sistema orientado a disponibilidad permitirá que cada región siga procesando compras con su última información local (aunque quizá el inventario global quede inconsistente por un rato). El usuario puede seguir añadiendo productos al carrito y finalizando compras (*el sitio no cae*), y luego cuando se restablezca la conexión se reconcilian las diferencias (por ejemplo, ajustando stock, pudiendo que algunos pedidos deban resolverse si se vendieron más ítems de los disponibles). Este enfoque asume que la **experiencia continua del usuario es prioritaria**, y que la inconsistencia resultante es manejable o aceptable temporalmente.

En contraste, *preferir consistencia* implica que en cuanto el sistema detecta que no puede garantizar un estado único (por desconexión de nodos), **restringe las operaciones**. Un ejemplo clásico es un **sistema bancario centralizado**: si la base de datos de cuentas queda particionada en dos mitades, es probable que el sistema opte por congelar ciertas transacciones o funcionar en modo lectura solamente hasta resolver la partición, en lugar de arriesgar dobles débitos o saldos incorrectos. Más cercano a la web, pensemos en **Google Cloud Spanner**: está diseñado como CP; ante un problema de red entre réplicas, Spanner preferirá degradar su disponibilidad (podría rechazar escrituras en una región) antes que permitir una divergencia de datos. Otro ejemplo es **Apache Zookeeper**: si pierde la mayoría de sus nodos (quórum), deja de responder consultas de estado en vez de servir datos potencialmente incoherentes. La idea es que para aplicaciones que requieren *precisión absoluta* (ej. coordinadores de tareas distribuidas, sistemas de configuraciones críticas, contabilidad financiera), es mejor estar brevemente indisponible que propagar datos incorrectos.

En términos de **¿qué se sacrifica cuando hay una caída?**:

- En un sistema que elige disponibilidad (tipo eventual/AP), **se sacrifica la consistencia** a corto plazo. Los usuarios pueden observar datos antiguos o inconsistentes entre distintas partes del sistema durante el fallo. La garantía es que el servicio sigue en pie y eventualmente las inconsistencias se resolverán una vez el fallo pase, pero no se evita que existan en el ínterin.
- En un sistema que elige consistencia (tipo fuerte/CP), **se sacrifica la disponibilidad** inmediata. Algunas operaciones se bloquearán o fallarán hasta que el sistema vuelva a un estado saludable donde pueda sincronizar todo. Los usuarios pueden experimentar errores o tiempo de espera, pero cualquier dato que reciban sigue respetando las reglas de consistencia establecidas (no se propagan escrituras conflictivas ni lecturas sucias).

Este compromiso es el corazón del Teorema **CAP**, que formalmente establece que en presencia de una partición de red, un sistema distribuido *no puede garantizar simultáneamente consistencia y disponibilidad*. Debe ceder en uno de ellos. Muchos sistemas modernos ofrecen configuraciones *tunable*: por ejemplo, **Cassandra** permite ajustar el nivel de consistencia (desde estricta hasta eventual) sacrificando más o menos disponibilidad/performance según la necesidad. **MongoDB** en un clúster replicado típico prefiere disponibilidad (los secundarios pueden responder lecturas

eventualmente consistentes), pero si se requiere, el cliente puede exigir lecturas solo del primario para tener consistencia más fuerte a costa de potencial menor disponibilidad de datos en ciertos casos.

Ejemplos reales de decisiones:

- **Redes sociales y contenidos web (Disponibilidad sobre consistencia):**
Imaginemos un sistema de timeline de Twitter. Prefiere estar siempre online: si una región queda aislada, los usuarios de esa región aún pueden tuitear localmente. Quizás cuando la red se restablezca habrá que reconciliar tweets (dos usuarios podrían haber reclamado el mismo nombre de trending topic, etc.), pero Twitter privilegiará que la plataforma siga usable durante la incidencia. Los usuarios podrían ver un conteo de “likes” desactualizado en algunos tweets, pero la aplicación no deja de funcionar.
- **Sistemas financieros o de inventario crítico (Consistencia sobre disponibilidad):** Un exchange de criptomonedas o bolsa de valores distribuida internacionalmente podría, ante problemas de comunicación, *detener temporalmente las operaciones* en ciertas regiones para asegurar que no haya discrepancias en precios o en posiciones de los clientes. Prefieren congelar movimientos (no disponible) antes que permitir arbitrajes indebidos o inconsistencias en los libros contables. Del mismo modo, una base de datos de inventario de almacén podría rechazar actualizar stock si no puede verificar el estado global, para no vender mercancía inexistente; tal decisión frena ventas (indisponibilidad parcial) pero garantiza consistencia en el registro.

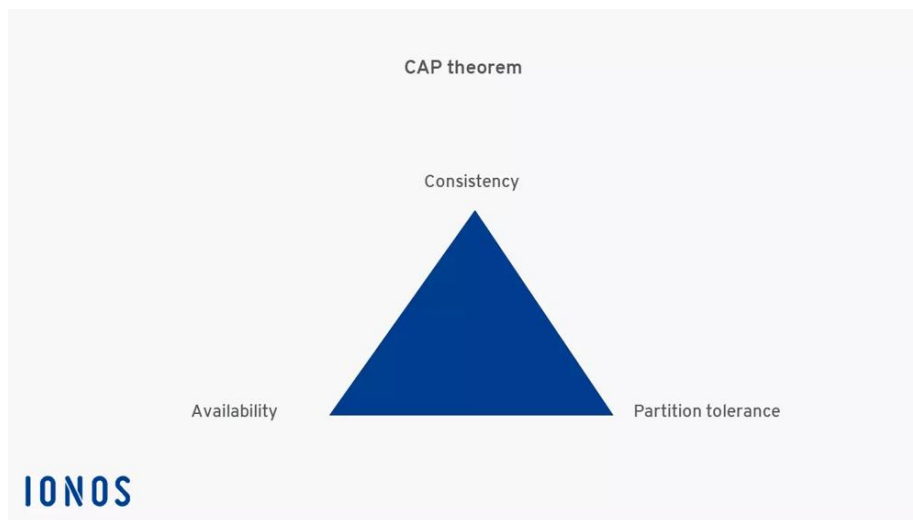
En resumen, **disponibilidad vs. consistencia es una decisión de ingeniería guiada por la naturaleza de la aplicación**. No es que a los ingenieros “no les importe” la consistencia en sistemas eventual, sino que han evaluado que *en su contexto* es mejor tener el sistema siempre operando y lidiar luego con posibles discrepancias. Por otro lado, en sistemas críticos, se asume que una respuesta incorrecta es peor que ninguna respuesta, por lo que se opta por esperar hasta obtener la verdad única. Este compromiso es introducido aquí de forma conceptual; más adelante, con el teorema CAP, formalizaremos cómo se define y se limita matemáticamente esta elección inexorable entre **consistencia y disponibilidad** en presencia de particiones. Lo importante es que el desarrollador comprenda que **no existe un “modelo de consistencia perfecto” sin costo**: cada modelo y cada elección refleja una preferencia sobre qué riesgo se tolera —¿datos momentáneamente inconsistentes o servicio momentáneamente interrumpido?— y esa decisión se toma en base a los requerimientos del sistema y las expectativas de los usuarios.

Referencias técnicas: Para profundizar, la documentación de bases de datos distribuidas ofrece más detalles sobre estos modelos. Por ejemplo, Amazon DynamoDB explica sus modos de lectura consistente vs eventual y su costo; Azure Cosmos DB describe cinco niveles de consistencia (fuerte, obsolescencia limitada, sesión, prefijo consistente, eventual) para ajustar este trade-off; Google Cloud Spanner muestra cómo logra consistencia fuerte global sacrificando disponibilidad en ciertos escenarios. Asimismo, investigaciones clásicas como “*Session Guarantees*” de Terry et al. discuten

las garantías de sesión (RYW, lecturas/escrituras monótonas, etc.), y por supuesto la formulación formal del teorema CAP por Gilbert y Lynch (2002) explora los límites teóricos de estos sistemas. En la práctica, ingenieros deben evaluar estos modelos al diseñar sistemas: **no hay solución única**, sino la correcta para el problema dado. La clave es que ahora el estudiante comprenda claramente **qué ofrece cada modelo, cuándo usarlos, y cómo se relacionan con la disponibilidad**, sentando las bases para entender las decisiones de arquitectura de los sistemas distribuidos modernos.

Introducción al Teorema CAP en sistemas distribuidos

El **Teorema CAP** (por *Consistency, Availability, Partition tolerance*) establece que un sistema distribuido, bajo condiciones de fallos de red, **solo puede garantizar dos de tres propiedades: Consistencia, Disponibilidad y Tolerancia a particiones**. En otras palabras, ante una partición de red es imposible mantener al mismo tiempo los datos consistentes y el sistema completamente disponible. Esta idea suele ilustrarse con la analogía “*barato, rápido y de buena calidad: seleccione dos*”, aplicada a la informática distribuida. El teorema fue propuesto inicialmente por Eric Brewer en 2000 y demostrado formalmente en 2002, por lo que también se le conoce como el *Teorema de Brewer*.



*Figura - Representación del “triángulo CAP”: en un sistema distribuido solo es posible lograr **dos** de estas tres propiedades a la vez (Consistencia, Disponibilidad, Tolerancia a particiones). Si ocurre una partición de la red, el sistema debe sacrificar Consistencia o Disponibilidad para poder seguir funcionando.*

En la era de las aplicaciones distribuidas y la nube, comprender CAP es fundamental. Todas las aplicaciones en la nube están respaldadas por sistemas distribuidos, es decir, datos replicados en múltiples nodos que cooperan. A continuación, introduciremos de forma intuitiva cada una de las propiedades C, A, P, y veremos por qué en situaciones de fallo de red solo podemos “elegir” dos de ellas al mismo tiempo. También presentaremos **analogías cotidianas** (cajeros automáticos, redes sociales, banca,

tiendas en línea) para interiorizar esta limitación, y revisaremos **ejemplos de sistemas reales** que priorizan distintas combinaciones (CA, CP, AP) según sus objetivos.

Las tres propiedades del teorema CAP

Consistencia (C) – *Consistency / Coherencia*

En el contexto de CAP, **Consistencia** (a veces llamada *coherencia de datos*) significa que *todos los nodos del sistema ven la misma información al mismo tiempo*. Cada lectura en cualquier nodo debe reflejar la escritura de datos más reciente realizada en el sistema. Para lograr esto, cuando un cliente escribe o actualiza un dato en un nodo, el cambio **se replica inmediatamente** en los demás nodos antes de considerarse confirmado. En resumen, la consistencia garantiza que sin importar a qué servidor o réplica se conecte un usuario, obtendrá la versión más reciente de los datos (como si hubiera una única copia centralizada actualizada al instante).

Disponibilidad (A) – *Availability*

La **Disponibilidad** implica que el sistema *siempre responde a las solicitudes de los clientes*, incluso si uno o varios de sus nodos fallan. Un sistema altamente disponible nunca se “cae” por completo: *cada petición recibe alguna respuesta válida*, sin excepción. Importante notar que disponibilidad no garantiza que la respuesta contenga los datos más actualizados, sino que el sistema **no deja de operar**. Por ejemplo, si un servidor está caído, otro atenderá la solicitud para que el usuario obtenga **algún resultado** en lugar de un error o tiempo de espera. La idea es maximizar el *uptime* del servicio: siempre hay una respuesta ante cualquier lectura o escritura, priorizando la continuidad del servicio.

Tolerancia a particiones (P) – *Partition Tolerance*

Una **Partición** de red ocurre cuando se interrumpe la comunicación entre componentes del sistema distribuido. Puede deberse a fallos de red, retrasos graves en las comunicaciones o caídas de enlaces que *dividen* el clúster en dos o más subconjuntos incomunicados. La **Tolerancia a particiones** significa que *el sistema sigue funcionando a pesar de estas fracturas en la red*. Es decir, incluso si los nodos quedan aislados en grupos, *cada partición continúa operativa* procesando solicitudes en la medida de lo posible. Diseñar con tolerancia a particiones implica asumir que los fallos de red son inevitables en sistemas distribuidos y preparar la aplicación para sobrevivir a ellos sin colapsar por completo.

El triángulo CAP: ¿por qué solo dos de tres?

El teorema CAP se suele resumir con la frase “*Consistencia, Disponibilidad, Particiones: elige dos*”. La razón detrás de esta restricción es que, **ante una partición de red (P), un sistema distribuido debe sacrificar o la consistencia (C) o la disponibilidad (A)** para mantener las otras propiedades. Mientras la red funcione bien (sin particiones), es

posible tener un sistema *consistente y disponible* al mismo tiempo. Sin embargo, cuando ocurre un corte de comunicación entre nodos, se presenta un **dilema**:

- Si queremos **Consistencia plena durante la partición**, todos los nodos deben ver los mismos datos. Para lograrlo, cualquier actualización o acción que no pueda sincronizarse con la otra mitad particionada del sistema debe ser *rechazada o bloqueada*. En otras palabras, se prefiere detener operaciones antes que permitir discrepancias. Esto mantiene la coherencia de la información pero **compromete la disponibilidad**, ya que algunas solicitudes de los usuarios no obtendrán respuesta hasta resolver la partición.
- Si queremos **Disponibilidad total durante la partición**, cada subdivisión aislada del sistema debe seguir respondiendo por su cuenta. Esto implica que ambas mitades atenderán peticiones aun sin comunicarse entre sí. El servicio permanece *activo* para los usuarios, pero las dos particiones pueden no tener exactamente los mismos datos en ese momento (cada una puede operar con información desactualizada relativa a la otra). Es decir, se acepta una **inconsistencia temporal** a cambio de mantener el sistema en marcha.

En resumen, **durante un fallo de red no se pueden garantizar a la vez la consistencia de datos y la disponibilidad 100%**. El sistema debe tomar una decisión de diseño: o bien esperar a que la comunicación se restablezca para seguir coherente (perdiendo disponibilidad en el ínterin), o bien seguir funcionando aislado en cada partición (entregando resultados potencialmente distintos o desactualizados, sacrificando consistencia). Dado que en entornos distribuidos reales *la posibilidad de particiones es inevitable*, el teorema CAP nos advierte que siempre existirá este compromiso (*trade-off*). No es una limitación arbitraria sino consecuencia de las físicas básicas de la comunicación distribuida.

Nota: Muchos sistemas modernos implementan soluciones intermedias para mitigar este dilema. Por ejemplo, se utiliza **consistencia eventual** en enfoques AP, lo que significa que tras la partición los datos *acabarán sincronizándose* y convergiendo al mismo valor una vez restaurada la comunicación. De igual forma, se puede optar por **disponibilidad parcial** (degradar ciertas funciones durante la falla) en vez de un apagón total o por mecanismos de reconciliación post-partición. Aún así, incluso con estos matices, el principio básico de CAP se mantiene: *no es posible evitar completamente tener que ceder en consistencia o disponibilidad cuando la red falla*.

Analogías cotidianas para entender CAP

Para ilustrar de forma intuitiva estas situaciones, veamos **analogías de la vida real** en sistemas conocidos y cómo se manifiesta el dilema CAP en cada caso:

- **Cajero automático (ATM):** Imaginemos un cajero bancario conectado a la base de datos central del banco. En condiciones normales (sin partición), el ATM verifica tu saldo en tiempo real y opera con consistencia y disponibilidad. Ahora supongamos que el cajero *pierde conexión* con el servidor central – es decir, ocurre una partición entre el cajero y el banco. El sistema tiene dos opciones:

- (CP) *Priorizar consistencia*: el cajero **rechaza la operación** o se pone fuera de servicio si no puede validar el saldo en tiempo real. Así evita inconsistencias como entregar dinero que no debería (no permite, por ejemplo, dos retiradas simultáneas que dejen el saldo negativo), pero con la consecuencia de no estar disponible para el usuario en ese momento. (AP) *Priorizar disponibilidad*: el cajero **sigue dispensando efectivo aun sin conexión**, quizás hasta un cierto límite predefinido, y registra la operación localmente. De este modo el servicio continúa activo (alta disponibilidad), pero se introduce una **inconsistencia temporal**: el balance del cliente no se actualiza inmediatamente en la base de datos central. Una vez reestablecida la conexión, habrá que reconciliar esa información (por ejemplo, descontar el dinero retirado del saldo real). Vemos que ambas estrategias tienen pros y contras: la primera garantiza datos correctos en todo momento (consistencia) a costa de interrumpir el servicio, y la segunda mantiene al usuario satisfecho en el instante (servicio continuo) a costa de actualizar los datos después (consistencia eventual).
- **Red social**: En plataformas como redes sociales, típicamente se valora mucho la rapidez de respuesta y la experiencia continua del usuario. Supongamos que un amigo publica una foto o te envía un mensaje, pero alguna parte de la red social está momentáneamente particionada (por ejemplo, dos centros de datos de la aplicación no se comunican bien). Un diseño **AP** sería el habitual: la red social **permanece disponible** para todos los usuarios; pueden seguir viendo posts, dando “me gusta” o comentando, incluso si no todas las operaciones se sincronizan inmediatamente entre los servidores. Podría ocurrir que temporalmente *un usuario vea un conteo de “me gusta” distinto al que ve otro* en otra región, o que un comentario recién hecho tarde unos instantes en aparecer para otros. El sistema ha privilegiado la disponibilidad y rapidez, **sacrificando la consistencia instantánea** (los datos están ligeramente fuera de sincronía). Sin embargo, suele implementarse **consistencia eventual**: tras un breve periodo, cuando la conectividad se normaliza, la información se replica y todos terminan viendo los mismos “me gusta” y comentarios actualizados. En este ejemplo cotidiano, la pequeña inconsistencia temporal es aceptable para los usuarios, comparada con la alternativa de que la red social hubiera estado inaccesible mientras se resolvía el problema (lo cual sería mucho más notorio y negativo en la experiencia de usuario).
 - **Sistema bancario (transacciones financieras)**: A diferencia de una red social, en un banco la **precisión de los datos es crítica**. Pensemos en transferencias electrónicas o pagos. Si diferentes sucursales bancarias o sistemas de pago quedan divididos por una partición de red, la prioridad suele ser **evitar cualquier descuadre contable**. Un banco típicamente optará por un enfoque **CP**: es preferible *pausar o rechazar temporalmente ciertas operaciones* antes que arriesgar incoherencias en las cuentas. Por ejemplo, si el sistema que valida los fondos del cliente no responde, quizás la transacción no se procese hasta confirmarlo, o ciertas funcionalidades de la banca en línea se desactivan momentáneamente con un mensaje de “servicio no disponible, inténtelo más tarde”. Esto implica sacrificar disponibilidad en pro de consistencia y tolerancia a fallos: los datos de las cuentas seguirán **consistentes y seguros**, aunque los clientes experimenten demoras o indisponibilidad en el servicio. La experiencia

de usuario se ve afectada (nadie quiere que su transferencia se retrase), pero desde la perspectiva del banco es peor permitir una inconsistencia (por ejemplo, sobregiros o dinero “duplicado”) que dar un servicio al 100% durante una partición. Este enfoque coincide con la idea de que en aplicaciones financieras la coherencia de los datos y la integridad tienen mayor peso que la disponibilidad absoluta.

- **Tienda en línea (e-commerce):** Imaginemos ahora una tienda en línea de alcance global. Estos sistemas suelen estar distribuidos en múltiples servidores (gestión de inventario, carritos de compra, pagos, etc.). Si ocurre una partición, por ejemplo si el servicio de inventario queda aislado del de checkout, el negocio debe decidir cómo proceder. Un enfoque posible es **AP**: la tienda **permanece en línea aceptando pedidos** aunque esté momentáneamente desactualizada respecto al inventario real. Así los clientes pueden seguir comprando (alta disponibilidad), pero podría suceder que dos clientes compren simultáneamente la *última unidad* de un producto debido a la falta de consistencia inmediata entre nodos. Luego habría que resolver esa inconsistencia (por ejemplo, contactando a uno de ellos para ofrecerle esperar reposición o un reembolso). Alternativamente, un diseño **CP** detendría ciertas ventas hasta sincronizar el inventario —por ejemplo, marcar un artículo como “*no disponible temporalmente*” si no puede verificar stock actualizado— evitando ventas duplicadas pero arriesgando perder compras en ese lapso. Muchos sistemas e-commerce buscan un equilibrio: mantienen el sitio **disponible** para navegación y pedidos tanto como sea posible, manejando después las posibles inconsistencias de stock, excepto en operaciones críticas (como procesar el pago) donde sí aseguran la consistencia con controles estrictos. De esta forma combinan tolerancia a particiones y alta disponibilidad en la mayoría de servicios al cliente, aceptando una ligera inconsistencia manejable en pos de no interrumpir las ventas.

Ejemplos de sistemas reales: combinaciones CA, CP y AP

Según el teorema CAP, los sistemas distribuidos **se clasifican en tres categorías básicas** basadas en cuál de las dos propiedades priorizan (y cuál sacrifican):

- **Sistemas CP (Consistencia + Tolerancia a particiones):** Garantizan que los datos se mantienen coherentes entre nodos incluso si hay fallos de red, pero para ello *sacrifican disponibilidad* cuando ocurre una partición. Durante el evento de partición, alguna parte del sistema podría dejar de aceptar operaciones para evitar discrepancias. Este enfoque es común en **sistemas financieros, bancarios o críticos**, donde es preferible dejar a algunos usuarios en espera que arriesgar un error en los datos. En términos de bases de datos, un ejemplo es **MongoDB** configurado con escritura en nodo primario: ante una partición, si un nodo queda desactualizado, el clúster lo inhabilita (no permite acceder a datos potencialmente incoherentes) hasta que se recupere la comunicación. Otro ejemplo famoso es **Google Spanner**, una base de datos distribuida de alta consistencia (usa relojes globales para mantener coherencia), que prioriza CP para aplicaciones como transacciones bancarias, donde todos los nodos deben

acordar el mismo estado antes de proceder. Un sistema CP ofrecerá datos fiables y consistentes, al costo de que en situaciones de fallo algunos servicios queden momentáneamente fuera de línea (no disponibles) hasta resolver el problema.

- **Sistemas AP (Disponibilidad + Tolerancia a particiones):** Están diseñados para *seguir operando pase lo que pase*, incluso durante particiones de red, lo que implica que *no pueden garantizar consistencia inmediata* de los datos en todos los nodos. En caso de partición, cada nodo o partición del sistema atiende las peticiones localmente, pudiendo responder con **datos antiguos o divergentes** respecto a otros nodos. Estos sistemas privilegian la experiencia continua del usuario y la resiliencia del servicio. Un ejemplo claro es el **DNS (Domain Name System)**, la “agenda” de dominios de Internet, que es un sistema distribuido global de tipo AP: siempre hay servidores DNS alternativos respondiendo (altísima disponibilidad), pero cuando se actualiza un registro de dominio, la propagación a todos los servidores lleva tiempo y durante horas o días diferentes clientes pueden ver direcciones antiguas o nuevas según a qué servidor consulten. En bases de datos NoSQL, **Apache Cassandra** es representativo de AP: no tiene un nodo maestro sino que permite escribir en cualquier réplica; ante fallos de red continua aceptando operaciones en todas las particiones, logrando disponibilidad total a cambio de posibles discrepancias temporales entre réplicas. Cassandra implementa *consistencia eventual*: tras la partición, los nodos se *resincronizan* rápidamente para reconciliar las diferencias y reparar inconsistencias. Muchos servicios web modernos (redes sociales, sistemas de mensajería, caches distribuidos) siguen este modelo AP, tolerando tener datos ligeramente obsoletos durante unos segundos o minutos, con tal de no dejar de brindar servicio.
- **Sistemas CA (Consistencia + Disponibilidad):** Ofrecen datos siempre coherentes y sistemas altamente disponibles **mientras** el entorno de red sea fiable, pero *no toleran particiones*: si ocurre una falla de comunicación importante, no pueden garantizar seguir cumpliendo ambas propiedades. En la práctica, un “sistema CA puro” distribuido es considerado **inviable** porque en algún momento los fallos de red sucederán. Por eso, cuando se habla de combinar consistencia y disponibilidad suele referirse a contextos **no distribuidos o de red muy controlada**. Un ejemplo de CA podría ser una **base de datos relacional tradicional** (tipo SQL) operando en un solo servidor o en un clúster centralizado con replicación síncrona: bajo condiciones normales, estas bases de datos proporcionan transacciones ACID totalmente consistentes y están siempre disponibles para las aplicaciones clientes. Sin embargo, si el servidor principal se aísla o se rompe la comunicación en la réplica, el sistema no sabría entregar respuestas sin arriesgar inconsistencia, y típicamente deja de estar disponible en ese caso (lo cual equivale a no tolerar la partición). Otro ejemplo citado teóricamente es el software de un **cajero automático en una red cerrada**: en un ambiente controlado (por ejemplo, dentro de una sucursal bancaria con conexión local estable), se puede asumir que no habrá particiones y así el cajero ofrece datos consistentes y alta disponibilidad a la vez. Pero en cuanto ese cajero pierde conexión externa, como vimos, ya no podría mantener ambas propiedades. En resumen, los sistemas CA existen esencialmente cuando se

descarta la posibilidad de fallos de red significativos (por diseño o suposición), o se limita el dominio a una única ubicación. En escenarios distribuidos reales, si usted necesita consistencia fuerte y también alta disponibilidad, deberá implementar estrategias de recuperación ante fallos (clústeres con conmutación por error, etc.) sabiendo que una partición grave igual provocará interrupción, o considerar si realmente debe repartir la carga entre múltiples ubicaciones.

En conclusión, el teorema CAP nos recuerda las *limitaciones fundamentales* en el diseño de sistemas distribuidos. No debemos verlo como una maldición insalvable, sino como una guía para tomar decisiones informadas: dependiendo de las necesidades de nuestra aplicación (consistencia rigurosa de datos vs. servicio ininterrumpido) optaremos por un equilibrio CP o AP principalmente, ya que la tolerancia a fallos de red es generalmente imprescindible. Un buen ingeniero de sistemas distribuidos evaluará estos trade-offs y posiblemente combinará técnicas (consistencias eventuales, retransmisión de cambios, degradación controlada del servicio) para acercarse lo más posible a las tres propiedades, sabiendo que no puede tenerlas *todas al mismo tiempo*. Entender CAP ayuda a **elegir la arquitectura de datos adecuada** – por ejemplo, escoger entre una base de datos relacional fuerte u otra NoSQL más flexible – según si el caso de uso demanda estricta fidelidad de la información (CP, como un banco) o máxima disponibilidad y tolerancia a fallos con datos “suficientemente buenos” (AP, como una red social). Así, al diseñar sistemas distribuidos robustos, siempre preguntémonos: *¿qué es más importante para esta aplicación durante un fallo de red inevitable: mantenerla operativa o mantenerla consistente?* Las respuestas a esa pregunta nos orientarán en la elección correcta de tecnologías y estrategias, teniendo siempre presente el triángulo CAP.

Referencias: Brewer (2000); Gilbert y Lynch (2002); IBM Cloud Education – *¿Qué es el teorema CAP?*; Q2BStudio – *Teorema CAP en sistemas distribuidos*; Tokio School – *Teorema CAP: guía práctica*; IONOS – *CAP theorem: coherencia, disponibilidad y tolerancia a la partición*; IBM Cloud – *MongoDB y Cassandra vs CAP*, entre otros.

Bibliografía técnica y académica usada:

- **Harrison, Guy** – *Next Generation Databases: NoSQL and Big Data*. Apress, 2015.
- **Kleppmann, Martin** – *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
- **Stonebraker, Michael** – Artículos sobre la evolución del modelo relacional y bases distribuidas.
- **Documentación oficial de tecnologías específicas:**
 - [MongoDB](#)
 - [Apache Cassandra](#)
 - [Redis](#)
 - [Neo4j](#)
- **Google Cloud y Amazon Web Services (AWS)** – Whitepapers sobre arquitecturas distribuidas, consistencia y replicación.

- **Teorema CAP** – Resumen de la formulación original de Eric Brewer (2000), y trabajos posteriores como “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services” (Gilbert & Lynch, 2002).